

# Machine Learning

BY

Adnan Amin, Ph.D.

Assistant Professor

# Model Evaluation & Validation

# Why Model Evaluation?

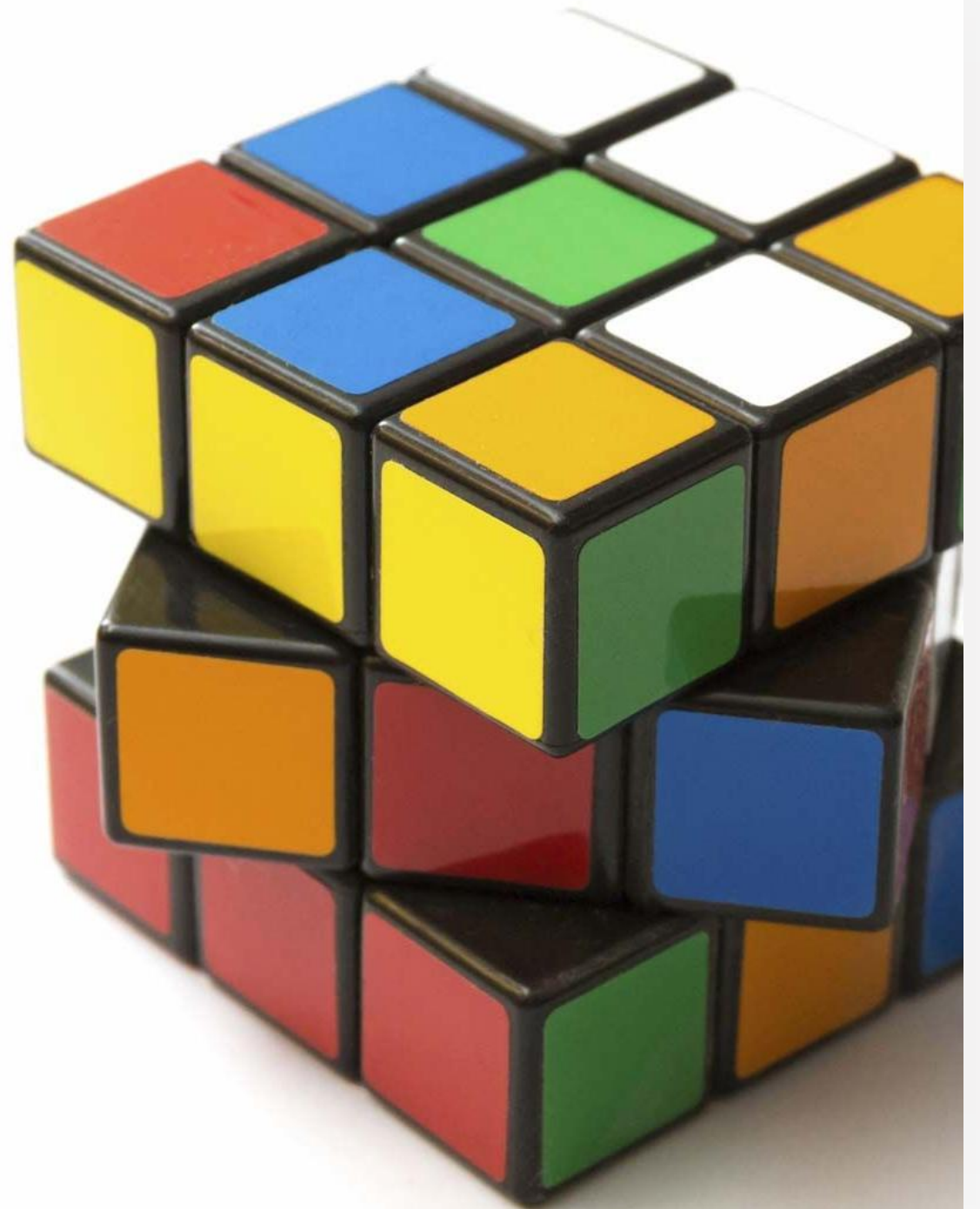
---

- To determine:
  - overlearned model
    - it could give us a good accuracy value for the training data but may do poorly on new (test) data.
  - Complex model
    - it could become hard to generalize or explain even though it performs well.
  - Measures of success and effectiveness of the model.
  - It could help us in comparison of various models.



# Confusion Matrix

## How To Use It & Interpret





A: accurate and precise



B: precise, but not accurate



C: neither accurate nor precise



D: accurate, but not precise

# Evaluation measures

- Confusion matrix
  - True positive
  - True negative
  - False positive
  - False negative
- Precision, recall, F1-Score, Accuracy

|                  |              | Actual Values |              |
|------------------|--------------|---------------|--------------|
|                  |              | Positive (1)  | Negative (0) |
| Predicted Values | Positive (1) | TP            | FP           |
|                  | Negative (0) | FN            | TN           |

## Confusion Matrix: Cont.

- Evaluating the performance of a machine learning model.
- A matrix of numbers that tell us where a model gets confused. It is a class-wise distribution of the predictive performance of a classification model.
- Types of Confusion Matrix:
  - Confusion Matrix for Binary Classes
  - Confusion Matrix for Multiple Classes







# PREDICTIVE VALUES

ACTUAL VALUES

POSITIVE (CAT)

NEGATIVE (DOG)

POSITIVE (CAT)

TRUE POSITIVE



3

YOU ARE  
A CAT

FALSE NEGATIVE



1

YOU ARE  
A DOG

TYPE II  
ERROR

NEGATIVE (DOG)

FALSE POSITIVE



2

YOU ARE  
A CAT

TYPE I  
ERROR

TRUE NEGATIVE



4

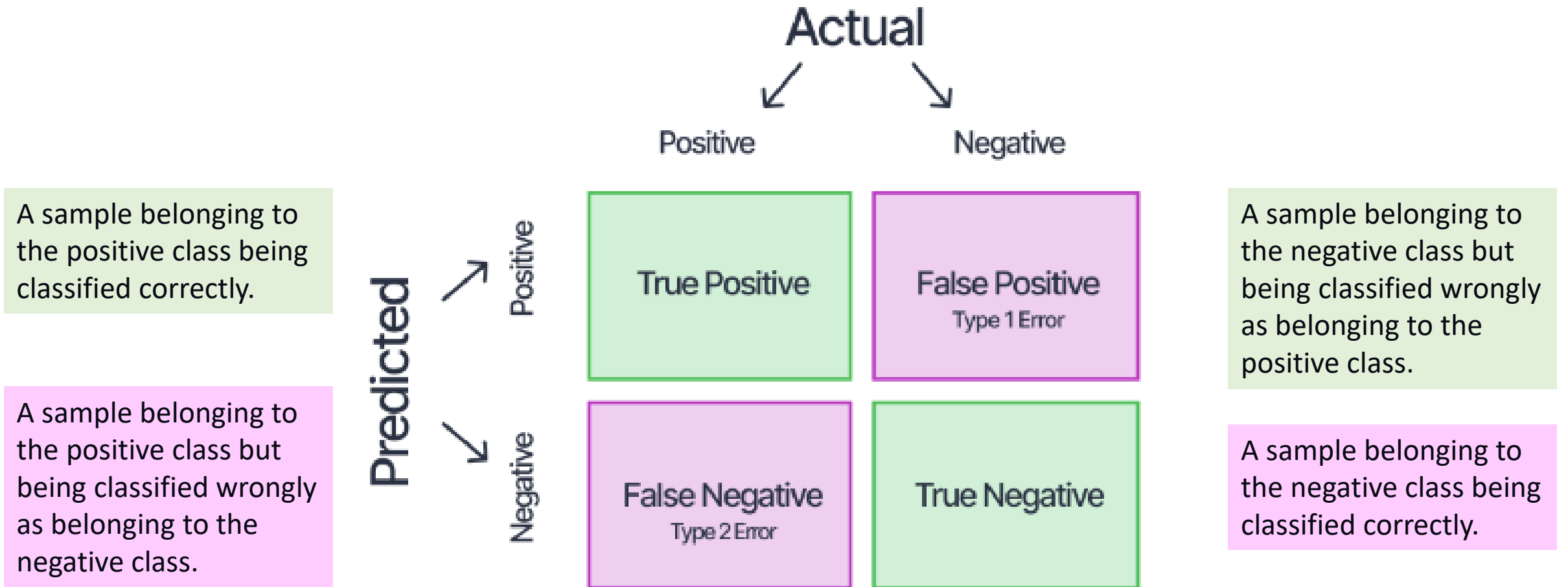
YOU ARE  
NOT A CAT

# Confusion matrix

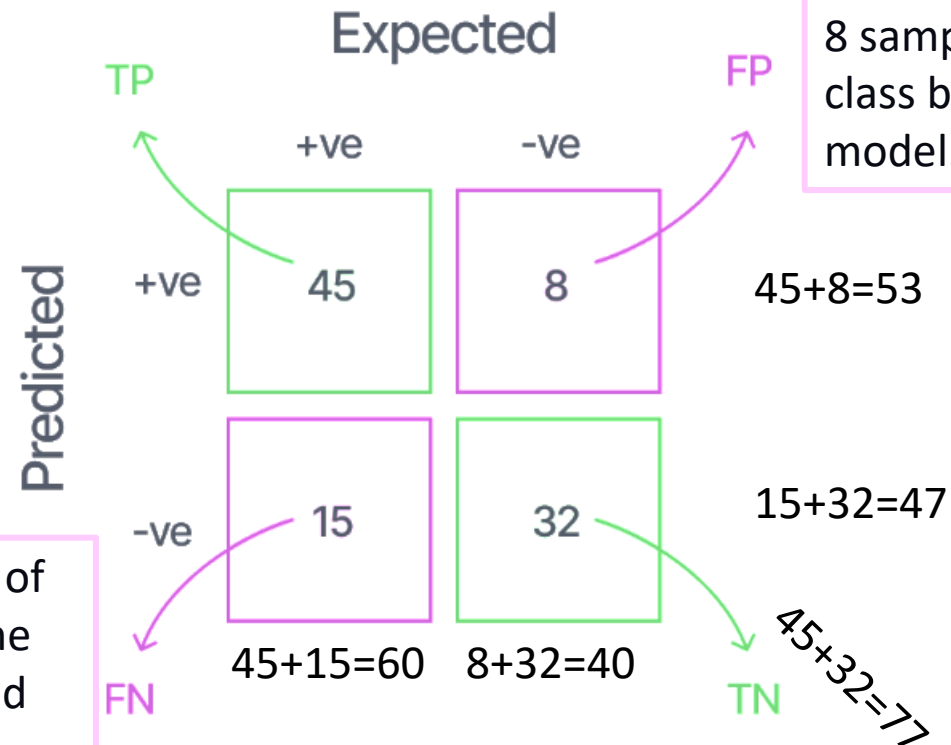
|                   | Predicted Positive  | Predicted Negative  |
|-------------------|---------------------|---------------------|
| Actually Positive | True Positive (TP)  | False Negative (FN) |
| Actually Negative | False Positive (FP) | True Negative (TN)  |

- **True Positive (TP):** This occurs when the model correctly predicts a sample as part of the positive class, which is its actual classification
- **False Negative (FN):** This happens when the model incorrectly classifies a sample from the positive class as belonging to the negative class
- **True Negative (TN):** This refers to instances where the model correctly identifies a sample as part of the negative class, which is its actual classification
- **False Positive (FP):** This occurs when the model incorrectly predicts a sample from the negative class as belonging to the positive class

# Confusion Matrix for Binary Classes



# Confusion Matrix Interpretation



8 samples were expected to be of negative class but were classified as “positive” by the model. They are thus called “False Positives.”

15 samples that were expected to be of the positive class were classified as the negative class by the model. It is called “False Negatives” because the model predicted “negative,” which was wrong.

The correct classifications are the diagonal elements of the matrix

# State-of-the-art Evaluation measures

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

The number of samples correctly classified out of all the samples present in the test set.

$$\text{Precision} = \frac{TP}{TP + FP}$$

(For Positive Class): The number of samples actually belonging to the positive class out of all the samples that were *predicted* to be of the positive class by the model.

$$\text{Recall} = \frac{TP}{TP + FN}$$

(For Positive Class): The number of samples predicted correctly to be belonging to the positive class out of all the samples that *actually belong* to the positive class.

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

(For Positive Class): The harmonic mean of the precision and recall scores obtained for the positive class.

$$\text{Specificity} = \frac{TN}{FP + TN}$$

The number of samples predicted correctly to be in the negative class out of all the samples in the dataset that actually belong to the negative class.

# Most common metrics in machine learning

- **True Positive Rate (TPR)** measures the proportion of actual positive examples correctly classified by the model:

$$\text{TPR} = \frac{\text{Positives classified correctly}}{\text{Total positives}} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **False Positive Rate (FPR)** measures the proportion of actual negative examples that are incorrectly identified as positives by the model:

$$\text{FPR} = \frac{\text{Negatives classified incorrectly}}{\text{Total negatives}} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

# F-beta score or F-measure:

- The F-beta score is a generalization of the F1 score. It is a weighted
- harmonic mean of precision and recall, where the beta parameter controls the relative importance of precision and recall.

$$F_{\beta} = \frac{(1 + \beta^2) \times (\textit{precision} \times \textit{recall})}{(\beta^2 \times \textit{precision}) + \textit{recall}}$$

- When beta (  $\beta$  ) is set to 1, the F1 score is obtained, which is the harmonic mean of precision and recall. The F-beta score is a useful metric for imbalanced datasets, where one class may be more important than the other.

For example, if we want to prioritize precision over recall for the minority class, we can set  $\beta < 1$ . To see why that's the case, set  $\beta = 0$  in the  $F_{\beta}$  formula, which implies  $F_{\beta} = \textit{precision}$ .

Conversely, if we want to prioritize recall over precision for the minority class, we can set  $\beta > 1$  (we can set  $\beta = \infty$  in the  $F_{\beta}$  formula to see it reduce to recall).



# Confusion Matrix for Multiple Classes

Suppose we have the test set (consisting of 191 total samples) of a dataset with the following distribution:

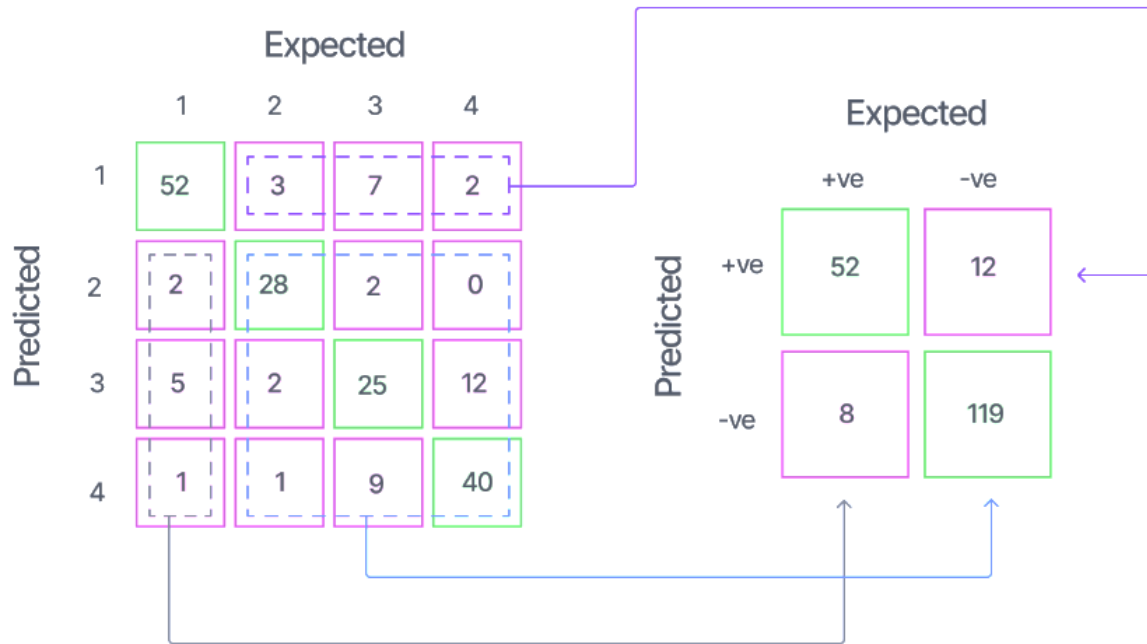
| Class | Nº of Samples |
|-------|---------------|
| 1     | 60            |
| 2     | 34            |
| 3     | 43            |
| 4     | 54            |

Assume the confusion matrix obtained by training a classifier and evaluating the trained model on this test set

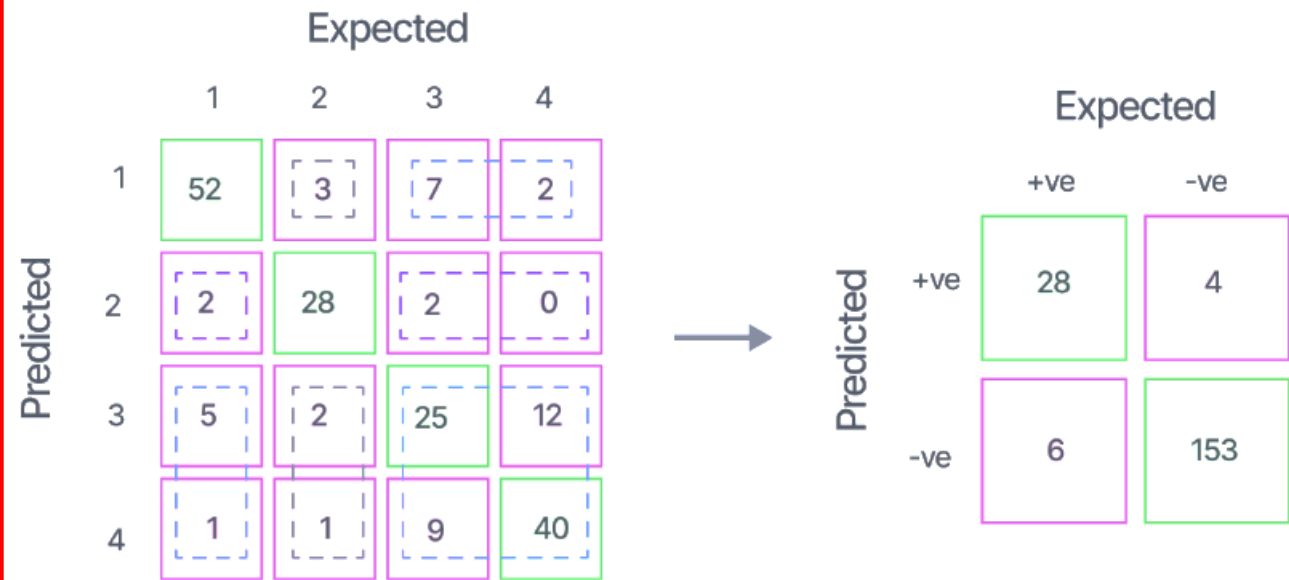
|           |   | Expected |    |    |    |
|-----------|---|----------|----|----|----|
|           |   | 1        | 2  | 3  | 4  |
| Predicted | 1 | 52       | 3  | 7  | 2  |
|           | 2 | 2        | 28 | 2  | 0  |
|           | 3 | 5        | 2  | 25 | 12 |
|           | 4 | 1        | 1  | 9  | 40 |

# Convert the confusion matrix into a one-vs-all.

one-vs-all matrix for class-1



one-vs-all matrix for class-2



# Performance Evaluation of the Model

| Class | Precision (%) | Recall (%) | F1-Score (%) |
|-------|---------------|------------|--------------|
| 1     | 81.25         | 86.67      | 83.87        |
| 2     | 87.50         | 82.35      | 84.85        |
| 3     | 56.82         | 58.14      | 57.47        |
| 4     | 78.43         | 74.07      | 76.19        |

# Micro F1-Score

- The micro-averaged f1-score is a global metric that is calculated by considering the net TP, i.e., the sum of the class-wise TP (from the respective one-vs-all matrices), net FP, and net FN. These are obtained to be the following:
- Net TP =  $52+28+25+40 = 145$   
Net FP =  $(3+7+2)+(2+2+0)+(5+2+12)+(1+1+9) = 46$   
Net FN =  $(2+5+1)+(3+2+1)+(7+2+9)+(2+0+12) = 46$
- Note that for every confusion matrix, the net FP and net FN will have the same value. Thus, the micro precision and micro recall can be calculated as:
- Micro Precision =  $\text{Net TP}/(\text{Net TP}+\text{Net FP}) = 145/(145+46) = 75.92\%$   
Micro Recall =  $\text{Net TP}/(\text{Net TP}+\text{Net FN}) = 75.92\%$
- Thus, Micro F-1 = Harmonic Mean of Micro Precision and Micro Recall = 75.92%.
- Since all the measures are global, we get:  
Micro Precision = Micro Recall = Micro F1-Score = Accuracy = 75.92%

# Macro F1-Score

- The macro-averaged scores are calculated for each class individually, and then the **unweighted** mean of the measures is calculated to calculate the net global score. For the example we have been using, the scores are obtained as the following:

| Class | Precision (%) | Recall (%) | F1-Score (%) |
|-------|---------------|------------|--------------|
| 1     | 81.25         | 86.67      | 83.87        |
| 2     | 87.50         | 82.35      | 84.85        |
| 3     | 56.82         | 58.14      | 57.47        |
| 4     | 78.43         | 74.07      | 76.19        |

The unweighted means of the measures are obtained to be:

Macro Precision = 76.00%

Macro Recall = 75.31%

Macro F1-Score = 75.60%

# Weighted F1-Score

The weighted-average scores take a sample-weighted mean of the class-wise scores obtained. So, the weighted scores obtained are:

$$\text{Weighted Precision} = \frac{81.25 \times 60 + 87.50 \times 34 + 56.82 \times 43 + 78.43 \times 54}{64 + 32 + 44 + 51} \% = 76.07\%$$

$$\text{Weighted Recall} = \frac{86.67 \times 60 + 82.35 \times 34 + 58.14 \times 43 + 74.07 \times 54}{64 + 32 + 44 + 51} \% = 75.92\%$$

$$\text{Weighted F1 - Score} = \frac{83.87 \times 60 + 84.85 \times 34 + 57.47 \times 43 + 76.19 \times 54}{64 + 32 + 44 + 51} \% = 75.93\%$$

# Python for Computing a Confusion Matrix

- Use scikit-learn library: `from sklearn.metrics import confusion_matrix`

```
>>> confusion_matrix(expected, predicted)
array([[52,  2,  5,  1],
       [ 3, 28,  2,  1],
       [ 7,  2, 25,  9],
       [ 2,  0, 12, 40]], dtype=int64)

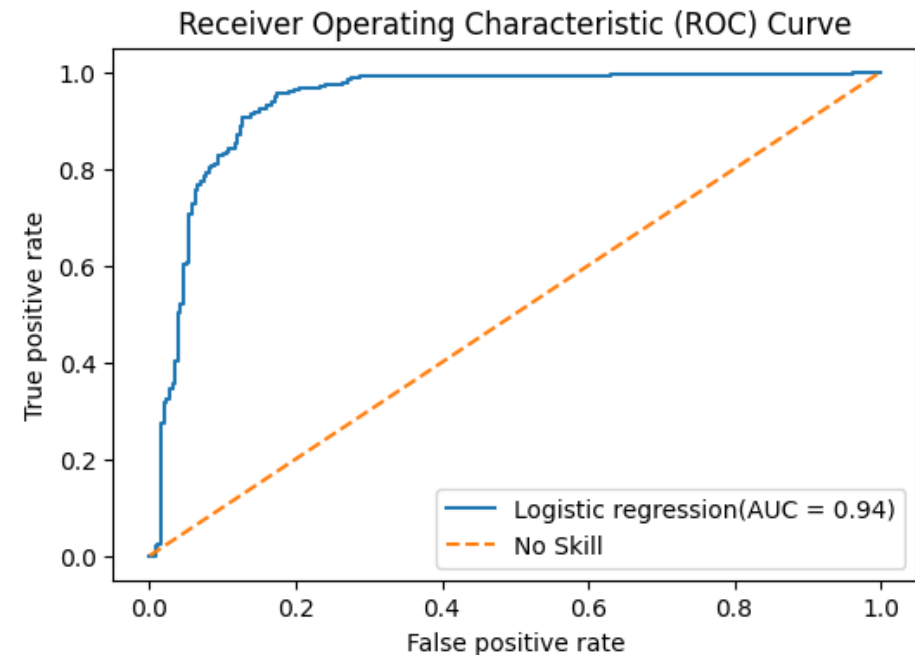
>>> print(classification_report(expected, predicted, digits=4))
              precision    recall  f1-score   support

     1         0.8125         0.8667         0.8387         60
     2         0.8750         0.8235         0.8485         34
     3         0.5682         0.5814         0.5747         43
     4         0.7843         0.7407         0.7619         54

 accuracy         0.7592         191
 macro avg         0.7600         0.7531         0.7560         191
weighted avg         0.7607         0.7592         0.7593         191
```

# Receiver Operating Characteristics

- **ROC** curves, are plots that display the **TPR** on the y-axis against the **FPR** on the x-axis for various threshold values.
- The ROC curve essentially represents the proportion of correctly predicted positive instances on the y-axis, contrasted with the proportion of incorrectly predicted negative instances on the x-axis.





# Reference

- <https://www.v7labs.com/blog/confusion-matrix-guide>